MODULE 05 · BLOCK 1 - COMPUTER PROGRAMMING AND PYTHON FUNDAMENTALS

Console input, output and type casting

print() with sep and end, why input() always gives text, and exactly which conversions succeed.

BLOCK 1 of 4 EXAM WEIGHT 18% MODULE 05 of 28

admin@sthithapragna.com

input() **always** returns a string. Nothing the user types becomes a number until your code converts it.

Any number of arguments, any types

```
1  print("A", 1, 2.5, True)     # A 1 2.5 True
2  print()                      # an empty line
3  print("x" + "y", 3 + 4)      # xy 7
```

1. Commas separate arguments. Each is converted to text and joined with one space.
2. With no arguments, print outputs just a newline.
3. Each argument is evaluated first, then printed.

sep goes between arguments; end goes after the last

```
1  print("x", "y", "z", sep="-")      # x-y-z
2  print("x", "y", "z", sep="")       # xyz
3  print("A", end="")
4  print("B", end="!\n")               # AB!
5  print(1, 2, sep="*", end=".\n")     # 1*2.
```

1-2 sep replaces the single space. It goes between arguments, never after the last one.

3-4 end replaces the newline. With end="", the next print continues on the same line.

Three rules, three traps

CODE	WHAT HAPPENS
`print("a", sep="-")`	a (one argument: nothing to separate)
`print(sep="-", "a")`	SyntaxError: positional argument follows keyword argument
`print(print("hi"))`	hi, then None: print returns None

> ON THE EXAM
>
> Keyword arguments like sep= and end= must come after all the values to print.

What happens when your program calls input()

01

Pause

The program stops and waits. Nothing below runs yet.

02

Prompt

If you passed a string, it is shown. No newline is added after it.

03

Type and Enter

The user types a line and presses Enter.

04

Return a str

input() returns the typed text as a str, without the newline.

ON THE EXAM

input("Age: ") prints the prompt itself. You do not need a separate print.

The user types 2, then 4

```
1  x = input()
2  y = input()
3  print(x + y)       # 24    joined, not added
4  print(x * 3)       # 222   repeated
5  print(x * y)       # TypeError: can't multiply by a str
```

3 Two strings: + joins them.

4 A string times an int repeats it.

5 A string times a string is not allowed at all.

Convert at the point of input

```
1  age = int(input("Age: "))
2  print("Next year:", age + 1)
```

1. Wrap input() in int() or float() so the variable holds a number from the start.
2. A comma in print mixes text and numbers safely. No str() needed.

If the user types 41, the screen shows: Age: 41, then Next year: 42.

Which conversions work

CALL	RESULT
`int("42") int(" 42 ")`	42. Surrounding spaces and newlines are ignored
`int("4_2") int("-7")`	42 and -7. Underscores and a sign are fine
`int("3.5") int("") int("12a")`	ValueError: not a whole-number string
`float("1e3") float(" 3.5 ")`	1000.0 and 3.5
`float("1,5")`	ValueError: the decimal separator is a point
`int(None)`	TypeError: wrong type, not a bad value

THE TIEBREAKER

A str with the wrong contents raises ValueError. A value of the wrong type raises TypeError.

int() truncates towards zero. // floors

```
print(int(3.9), int(-3.9))      # 3 -3    drop the fraction
print(3.9 // 1, -3.9 // 1)      # 3.0 -4.0  round down
print(round(3.9), round(-3.9))  # 4 -4    nearest
```

1. int() simply throws the fraction away, so negatives move towards zero.
2. Floor division always moves down, so −3.9 becomes −4.

Converting to text and to truth values

```
1  print(str(10) + str(5))      # 105
2  print(str(True), str(0.5))   # True 0.5
3  print(bool("False"))         # True   a non-empty string
4  print(int(True), float(False))   # 1 0.0
5  print(str(bool(1) + 12 / 2)) # 7.0
```

1. str() turns anything into text, so + now joins instead of adding.
3. bool() of a string only asks whether it is empty.
5. Work inside out: True + 6.0 is 7.0, then str gives the text 7.0.

Mixing text and numbers in print

PLUS

Only joins strings

```
1  total = 5
2  print("Total: " + total)
```

TypeError: can only concatenate str (not "int") to str

COMMA OR STR()

Both work

```
1  total = 5
2  print("Total:", total)
3  print("Total: " + str(total))
```

Total: 5, twice. The comma adds the space for you.

ON THE EXAM

Python never converts types silently for +. A comma in print, or an explicit str(), avoids the error.

Older bank favourites: sys.argv and eval()

- **from sys import argv** gives the command-line words as a list of **strings**.
- python index.py 1 2 gives argv == ['index.py', '1', '2']. **argv[0] is the script name.**
- So argv[1] + argv[2] is '12', not 3. The same trap as input().
- **eval(input())** runs the typed text as Python: 3 gives an int, 1, 2 gives a tuple. Never use it on untrusted input.

f-strings: you will see them, the exam will not

- **f"Total: {total}"** puts values straight into text. They arrived in Python 3.6.
- Python 3.12 relaxed the rules: quotes inside the braces can match the outer ones.
- PCEP-30-02 tests print() with commas, sep, end and str(). Answer in those terms.

What to carry into the exam

- **input() returns a str.** "2" + "4" is "24"; "2" * 3 is "222".
- sep goes between arguments only. end replaces the final newline.
- print() returns None. Keyword arguments go after the values.
- Bad contents in a str: **ValueError**. Wrong type: **TypeError**.
- int(-3.9) is -3; -3.9 // 1 is -4.0.

The user types 3 at the first prompt and 2 at the second.

What is the output of this program?

```
1  x = input()
2  y = int(input())
3  print(x * y + x)
```

A 9

B 333

C The code raises a TypeError

D 32

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = input()
2  y = int(input())
3  print(x * y + x)
```

x is "3", y is 2. "3" * 2 is "33", plus "3" is "333".

A 9

Treats x as a number. Only y was converted.

B 333

str * int repeats, then str + str joins.

C The code raises a TypeError

*str * int is legal, and str + str is legal.*

D 32

*Joins x and y. The * happens first, and repeats x.*

THE SIGNAL

Track the type of every variable: only the values wrapped in int() or float() are numbers.

What is the output of this program?

```
1  print("a", "b", sep="", end="-")
2  print("c", "d", sep="+")
```

A `ab-c+d`

B `a b-c + d`

C `ab-\nc+d`

D `ab-c+d+`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print("a", "b", sep="", end="-")
2  print("c", "d", sep="+")
```

A `ab-c+d`

No gap between a and b, - instead of a newline, + between c and d.

B `a b-c + d`

Keeps the default space. sep replaces it.

C `ab-\nc+d`

end replaces the newline. It does not add to it.

D `ab-c+d+`

sep goes between arguments, never after the last.

THE SIGNAL

sep only between arguments; end only once, at the very end, replacing the newline.

> Each call is evaluated on its own.

Which two calls raise a ValueError? (Choose two.)

A `int(" 7 ")`

B `int("7.0")`

C `float("7e1")`

D `int("seven")`

◆ PAUSE AND DECIDE

Which two calls raise a ValueError? (Choose two.)

A `int(" 7 ")`

Surrounding spaces are stripped. Returns 7.

B `int("7.0")`

int() of a str needs a whole number. A decimal point is not allowed.

C `float("7e1")`

float() accepts E notation. Returns 70.0.

D `int("seven")`

Words are not digits.

THE SIGNAL

int() of a str accepts digits, one sign, underscores and outer spaces. Nothing else.

What is the output of this program?

```
1  x = -7.5
2  print(int(x), x // 1, round(x))
```

A -8 -8.0 -8

B -7 -7.0 -7

C -7 -8 -8

D -7 -8.0 -8

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = -7.5
2  print(int(x), x // 1, round(x))
```

round(-7.5): an exact half goes to the even neighbour, -8.

A `-8 -8.0 -8`
int() does not round down. It drops the fraction: -7.

B `-7 -7.0 -7`
Floor division moves down the number line, to -8.

C `-7 -8 -8`
-7.5 is a float, so // gives a float: -8.0.

D `-7 -8.0 -8`
Truncate, floor as a float, round half to even.

THE SIGNAL

For negatives: int() goes towards zero, // goes down, round() goes to the nearest.

The user types 12 at the first prompt and 3.5 at the second.

What is the output of this program?

```
1  a = input()
2  b = float(input())
3  print(int(a) + int(b), a + str(b))
```

A `15.5 123.5`

B The code raises a ValueError

C `15 123.5`

D `15 15.5`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = input()
2  b = float(input())
3  print(int(a) + int(b), a + str(b))
```

int("12") + int(3.5) is 12 + 3. "12" + "3.5" is "123.5".

A `15.5 123.5`
int(b) drops the .5 before the addition.

B The code raises a ValueError
b is already a float, and int() of a float is always fine.

C **`15 123.5`**
12 + 3 is 15; then two strings are joined.

D `15 15.5`
a + str(b) joins two strings. Nothing is added.

THE SIGNAL

int("3.5") would fail. int(3.5) is fine. Check whether the argument is a str or a float.

What is the output of this program?

```
1  x = print("Hi", end=" ")
2  print(x)
```

A Hi None

B Hi

C Hi\nNone

D None Hi

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = print("Hi", end=" ")
2  print(x)
```

A `Hi None`

print returns None; the space from end keeps both on one line.

B `Hi`

x does hold a value: None, which print shows as the word None.

C `Hi\nNone`

end=" " replaced the newline, so no new line starts.

D `None Hi`

Line 1 runs first, so Hi is printed first.

THE SIGNAL

print() shows text and returns None. Assigning its result gives you None, not the text.